

Task Manager Report:

1. Introduction to the Project:

This project covers the implementation of a simple functional Task Manager using both Dart and Flutter. It aimed to illustrate the usage of the core programming constructs learned within the course, but also to study how Dart works as a standalone programming language and how it's used in conjunction with Flutter in the development of real applications. The clear steps from basic logic through to a full app are demonstrated here with the building of the console version first, then expanding into a mobile-friendly Flutter UI.

2. Purpose and Motivation:

We decided upon Dart because it is clean, modern, and relatively easy to pick up, especially when coming from languages previously studied in the class, such as Java. The reason we chose Flutter is that it lets us take this Dart code and build a visual interface on top of it. This was a motivating factor in the desire to create something practical, aesthetic, and in line with current development tools in high demand. This project gave us the opportunity to not only learn how code works behind the scenes but also how it actually translates into a user's experience.

3. Application Overview

The application is a simple Task Manager for adding, showing, and deleting tasks. The Dart console version handles tasks by inputting text and printing the results, which allows one to easily see how the logic works. In the Flutter version, the same logic is applied but is represented in a modern interface where users can type a task into a text field, tap an add button, and remove them with a delete icon. In this combination, one can see a complete picture of how data flows from backend logic to frontend presentation.

4. Programming Constructs Implemented

The project uses heavy amounts of the fundamental constructs of programming: variables, lists, classes, functions, object-oriented programming, and conditionals. We designed a Task class in Dart that models a real-world object. We used a list to store multiple tasks and functions to add or delete them. Conditionals were applied to ensure tasks could safely be removed. Flutter added another layer by using widgets, state management with `setState`, constructors, and UI components. Herein lies evidence of how this course's theoretical knowledge applies directly to actual software development.

5. Language Comparison: Dart vs Java

While Dart and Java share similarities such as strong typing, object-oriented structure, and class-based design, they differ in subtle but meaningful ways. Generally, Dart is more readable than Java, with its concise syntax and fewer structural requirements. It also feels more flexible, especially when writing UI or asynchronous operations. Java, on the other hand, is known for its stability and reliability but can be more verbose. In terms of speed, both are optimized, but Dart really shines in UI environments because it compiles directly to native code using Flutter. Overall, Dart is easier to write and nicer for quick development, whereas Java still keeps its power for large enterprise applications.

6. Progression of the project

We started the project by writing the logic-only version in dart. This helped us isolate the logic from the presentation: creating tasks, assigning ids and handling data. When that was running in console we passed on to flutter. Flutter was about creating the interface, arranging the widgets and linking up the UI to the dart logic. We used a stateful widget which allowed us to update the screen every time a new task was added or deleted. This clean separation of logic from presentation helped us greatly debug and organize the project.

7. Problems and Solutions

The first problem we faced was how to correctly structure files in dart and flutter, given that flutter generates folders and files automatically. We also had trouble with figuring out why some imports simply didn't work, which we found out was because the flutter project was not created correctly from the command line. The last problem that we faced was how to manage state in flutter, how to update the UI when we change the data. We solved those problems by trial and error, documentation and testing. We learned how to create a correct flutter project, how to link the files correctly, how `setState()` updates the UI properly.

8. Final Result

The result is a functional Task Manager application in two forms: one for the console and one as a mobile UI. The Dart console app clearly illustrates the fundamentals of programming, whereas the Flutter app takes those fundamentals into the user interface. It fulfills all the requirements of the project, shows real understanding, and serves as a full example of how concepts from class evolve into functional software.

9. Individual Contributions

Each of the team members played an important role: one focused on writing and testing the Dart logic, another took care of setting up the Flutter project, designing the UI, and

connecting the logic to the interface; others contributed to debugging, researching Flutter concepts, drafting the documentation, and preparing the presentation. This balanced the work among team members so that the entire project was complete and reflected everyone's learning.

10. Conclusion

This project was not only an opportunity to demonstrate programming skills but also a chance to explore real development workflows. By using both Dart and Flutter, we learned how a single language can power both backend logic and modern app interfaces. The challenges faced strengthened our understanding, and the final product reflects the growth achieved throughout the course.